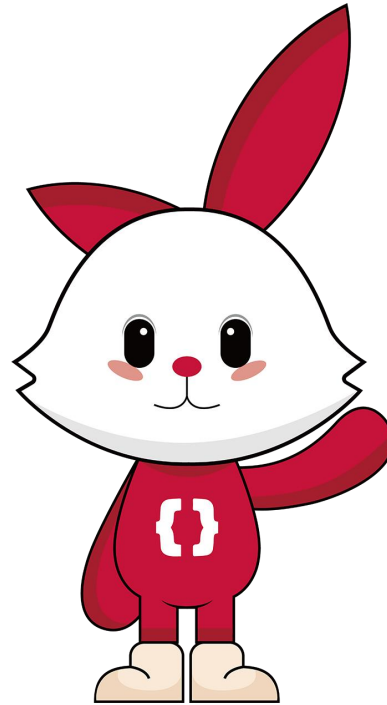




TSCINBUNY
ZYC0208



Tutorial introduction

This tutorial includes 3 files as shown below to help you learn in-depth electronics and programming knowledge:

“1_Get_start”: This folder stores the robot assembly guide and necessary software environment files. In order to complete the assembly accurately and quickly, please be sure to read it in detail and assemble it according to the manual;

1_Get_start-开始	2025/1/17 17:39	文件夹
2_Code-代码文件	2025/1/17 17:39	文件夹
3_APP	2025/1/18 10:02	文件夹

“2_Code”: This folder is used to store code files, and each code file is uploaded and used independently;

“3_APP” : This folder stores the APP installation package;



Table of contents

Tutorial introduction	2
1 Bluetooth control code burning	5
2 Bluetooth remote control APP	9
2.1 Install and set up APP	9
3 The first program code-Blink	11
3.1 About this lesson:	11
3.2 Main control board	11
3.3 Expansion board	12
3.4 Make your own “Blink” sketch	14
4 Motor test	19
4.1 Course Introduction	19
4.2 How the L293D motor driver chip works	20
4.3 Wiring diagram	20
4.4 Code analysis	21
5 Follow car	23
5.1 Course Introduction	23
5.2 Component introduction	23
5.3 How does ultrasound work?	24
5.4 Principle of following the car	25
5.5 Code analysis	25
5.6 Wiring diagram	28
6 obstacle avoidance car	29
6.1 Course Introduction	29
6.2 Principle of obstacle avoidance	29
6.3 Code upload	29
7 tracking car	31
7.1 Three-way tracking module	31
7.2 Tracking module wiring diagram	32



7.3 Code analysis	33
8 Servo mechanical arm	36
8.1 Course Introduction	36
8.2 Introduction to steering gear	36
8.3 Code analysis	37
8.4 Wiring diagram	40
9 Comprehensive control	41
9.1 Course Introduction	41
9.2 Bluetooth comprehensive control code analysis	41



1 Bluetooth control code burning

Make sure you have followed the instructions in the 1_Get_start folder to create your programming environment.

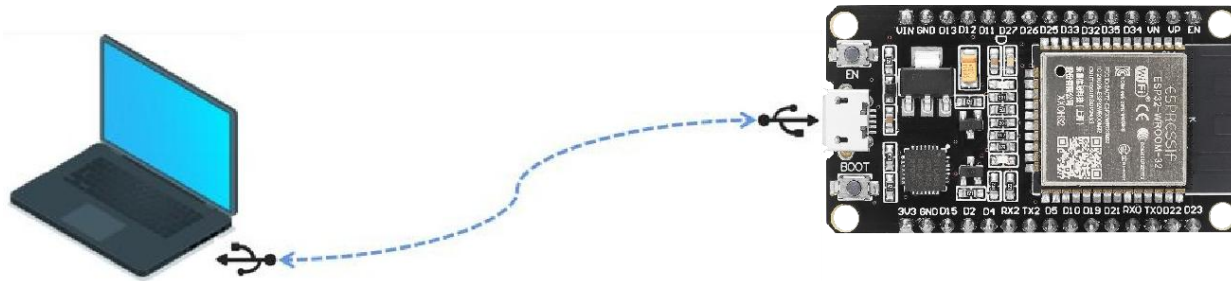
1_Get_start-开始	2025/1/17 17:39	文件夹
2_Code-代码文件	2025/1/17 17:39	文件夹
3_APP	2025/1/18 10:02	文件夹

Open the code file (folder path: \2_Arduino_Code-code file\0_ESP32-arm\0_ESP32-arm.ino)

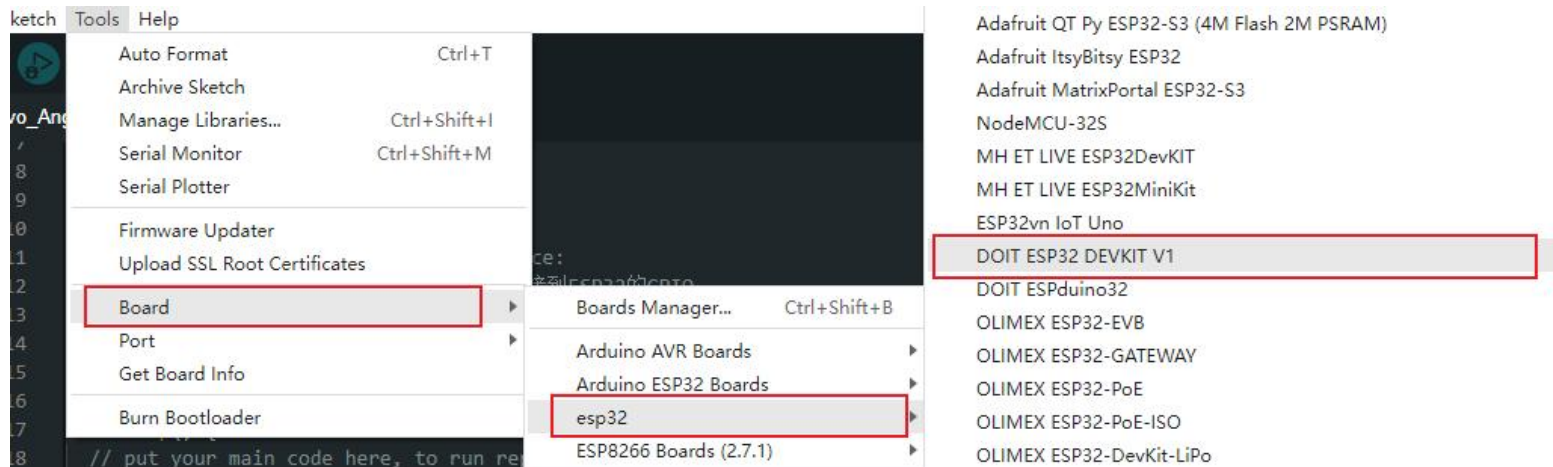
0_ESP32-arm	2024/11/30 17:27	文件夹
1_Motor_test	2024/11/28 16:35	文件夹
2_Ultrasonic_follow	2024/11/30 17:22	文件夹
3_Ultrasonic_obstacle_avoidance	2024/11/30 17:22	文件夹
4_Infrared_patrol_line	2024/11/30 17:22	文件夹
5_Robot_arm	2024/11/30 17:21	文件夹



Connect the motherboard to the computer using a USB cable



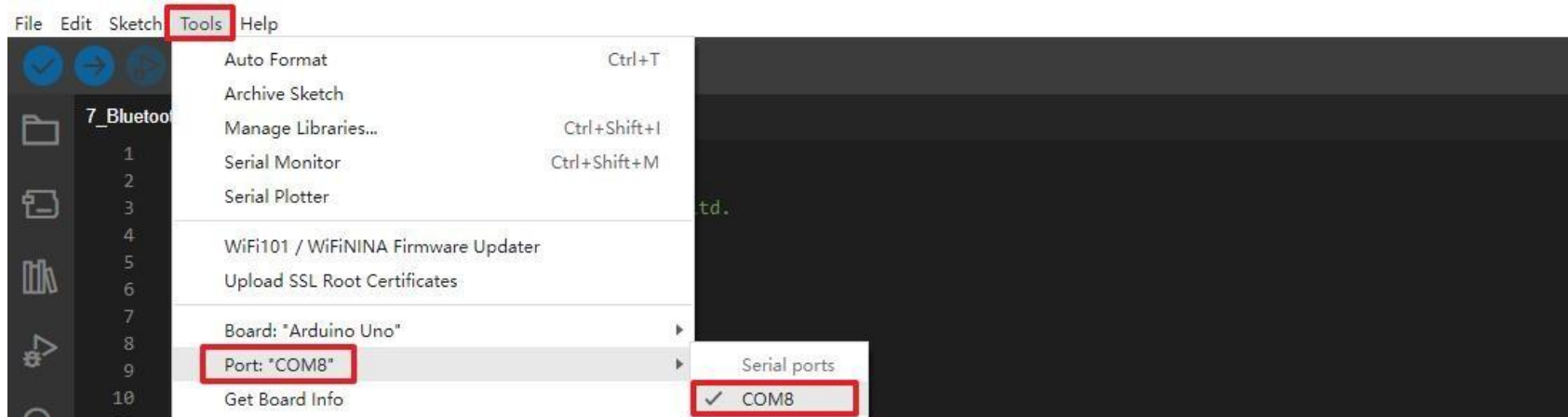
Open the "Tools" menu option, click "Development Board", and select the board type DOIT ESP32 DEVKIT V1.



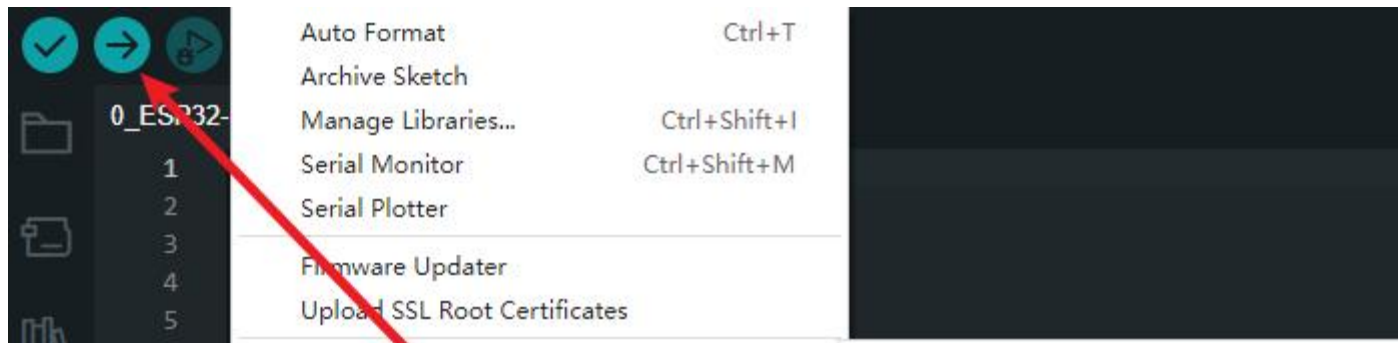
Then select the serial port COM number, here it is COM8, but the actual COM number will be different for



everyone.



Click the "upload" button to compile and upload the code.



At the bottom of the editor, you can see that the upload and writing is 100% complete. If an error or failure occurs, please check whether the board type and serial port number are correct.



```
Writing | ##### | 100% 0.83s  
avrdude: 5418 bytes of flash written  
avrdude done. Thank you.
```

Done uploading.

Ln 18, Col 27 Arduino Uno on COM8 3

Next, start the installation and settings of the Bluetooth APP, please see the following section.



2 Bluetooth remote control APP

2.1 Install and set up APP

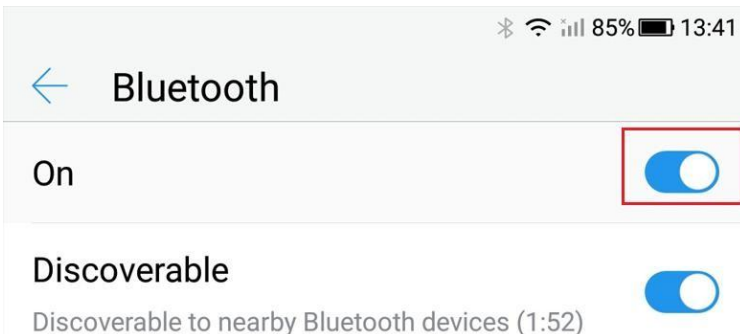
Open the directory 3_APP, install "TSCINBUNY.apk" to the Android phone and open it.

名称	修改日期	类型	大小
TSCINBUNY.apk	2025/1/18 10:01	APK 文件	40,912 KB

Open the App Store on the ios device, search and install TSCINBUNY



Next, we use an Android phone to demonstrate how to control the car through this application: First, turn on the Bluetooth function switch of the phone.





Open the APP you just installed and make sure the robot car is powered on.



Select the car model



control the main interface



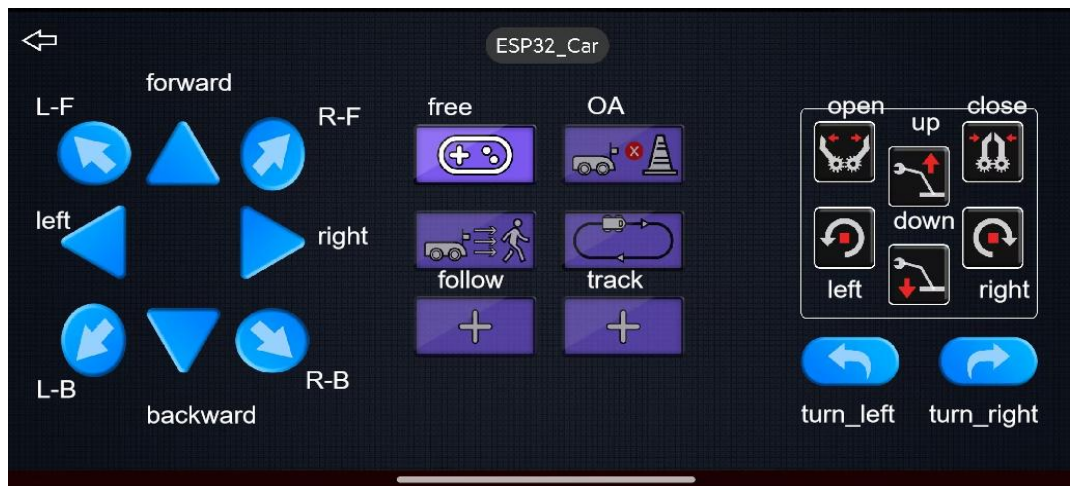
search for nearby devices



connect the device



operate



The following lessons will introduce and guide you in learning this kit in detail!



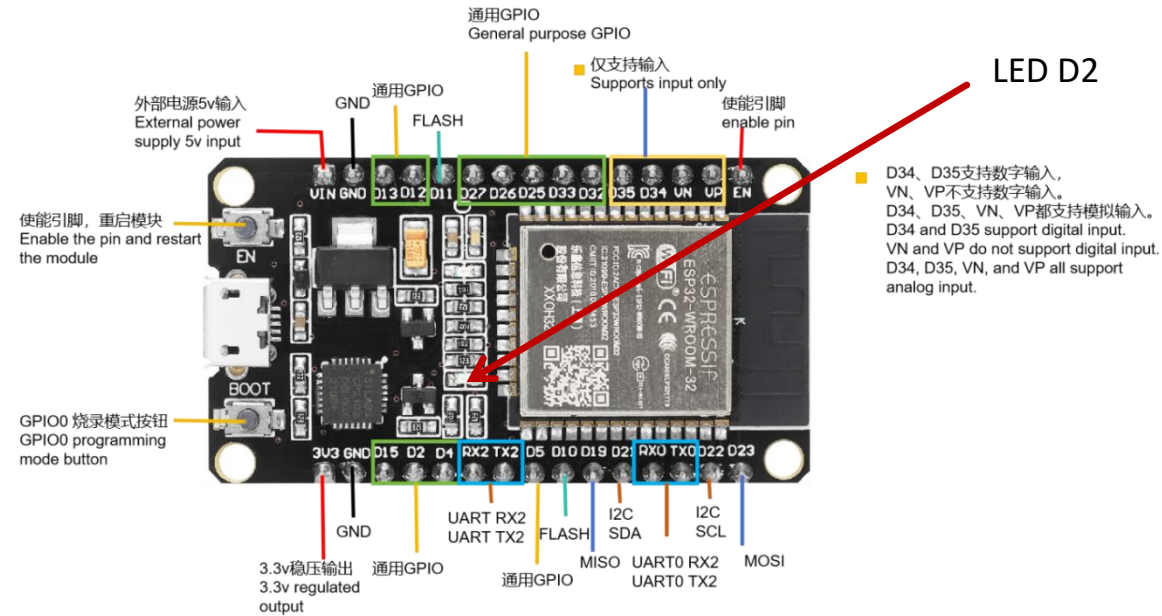
3 The first program code-Blink

3.1 About this lesson:

In this lesson, you'll learn how to program your control board to blink the built-in LED, as well as learn the basic steps for downloading the program.

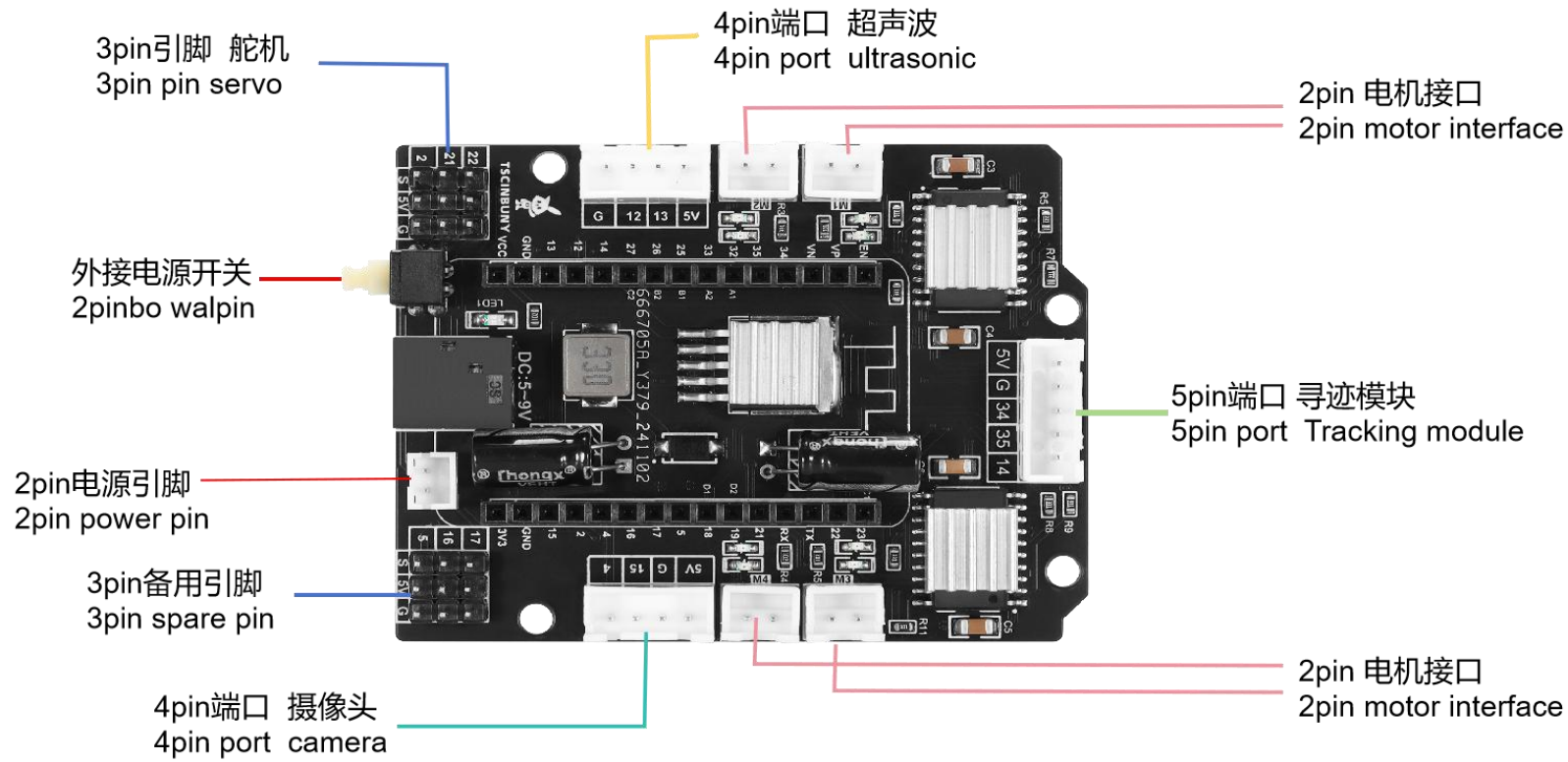
3.2 Main control board

This set provides an ESP32 motherboard, which integrates a Bluetooth module and a WiFi module. Its expansion board has well packaged and lead-out pins, such as the ultrasonic module pins. It also has an LED that you can control from the program sketch. it.



When you connect the motherboard to your computer with a USB cable, you may notice that its LED has blinked. This is because the board comes with the "Blink" sketch pre-installed.

3.3 Expansion board

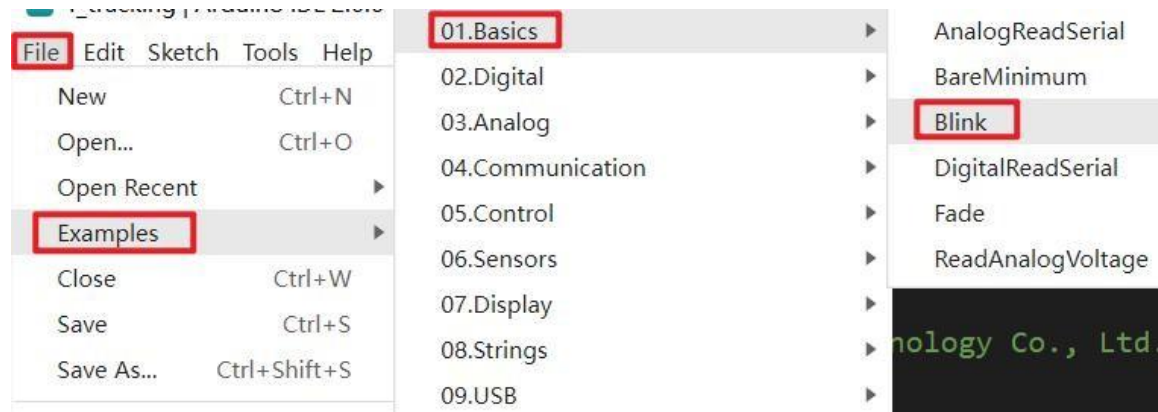




3.4 Make your own “Blink” sketch

In this lesson we will reprogram the board using our own Blink sketch and then change its blink rate. Now keep the board connected to your computer, set up the Arduino IDE and make sure you can find the correct serial port, and upload the program for testing.

The Arduino IDE includes a number of example sketches that you can load and use, including a “blink” example sketch for making an “LED”. The “Blink” sketch you will find in the IDE’s menu system File > Examples > 01.Basics.

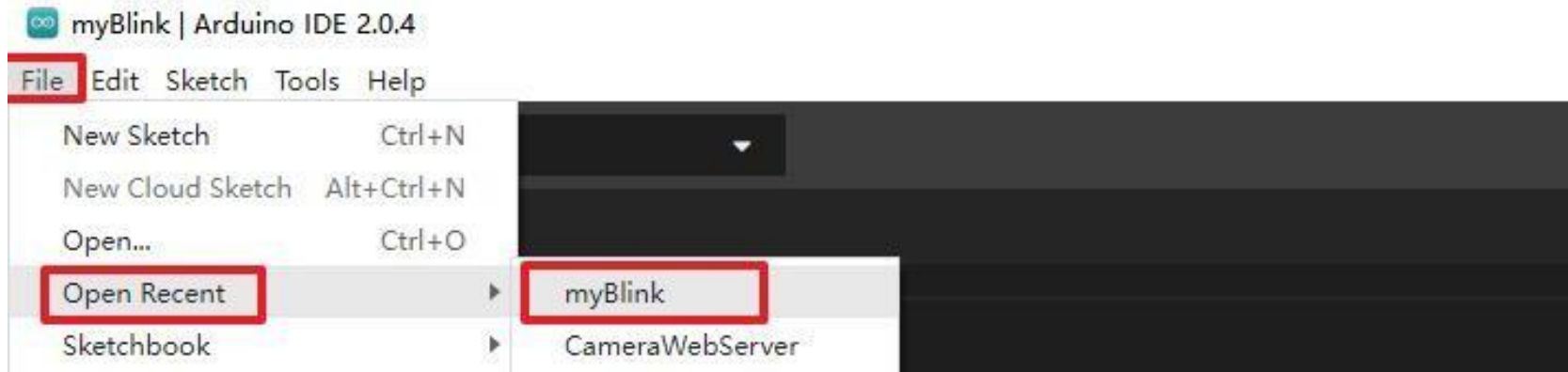


The example sketches included with the Arduino IDE are “read-only”. That is, you can upload them to the dashboard, but if you change them, you cannot save them as the same file. So the first thing you need to do is save your own copy.

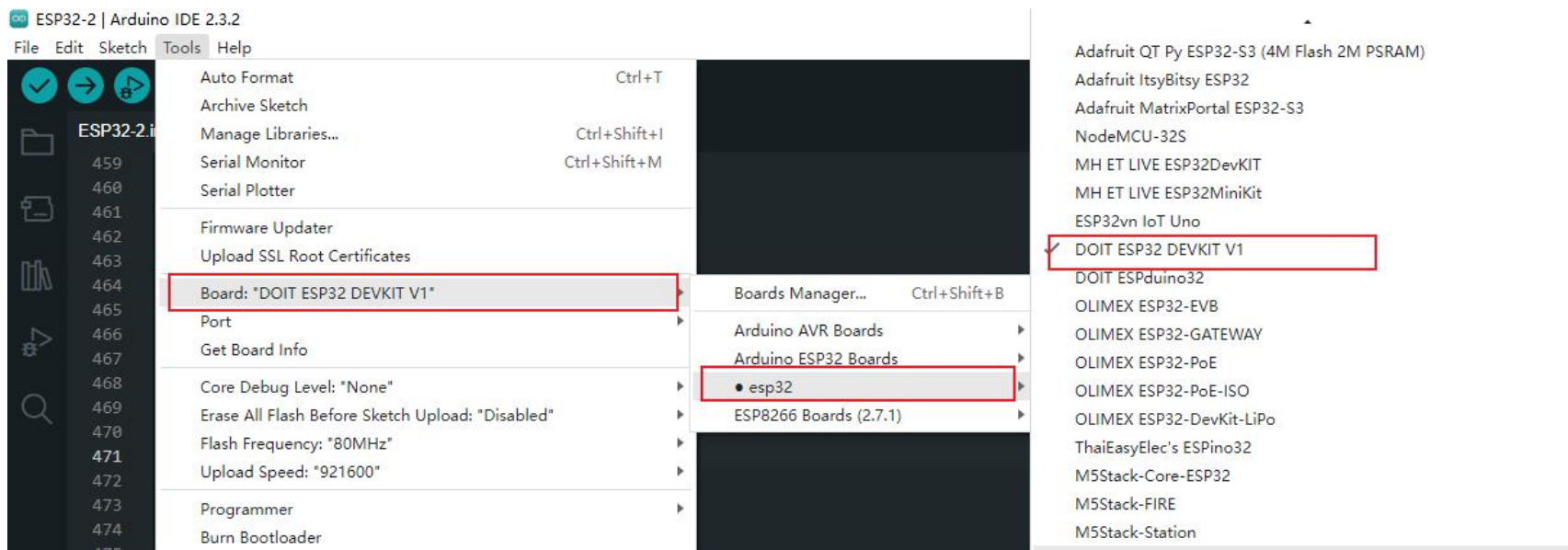
From the Arduino IDE’s File menu, select “Save As...” and save the sketch as “MyBlink”.

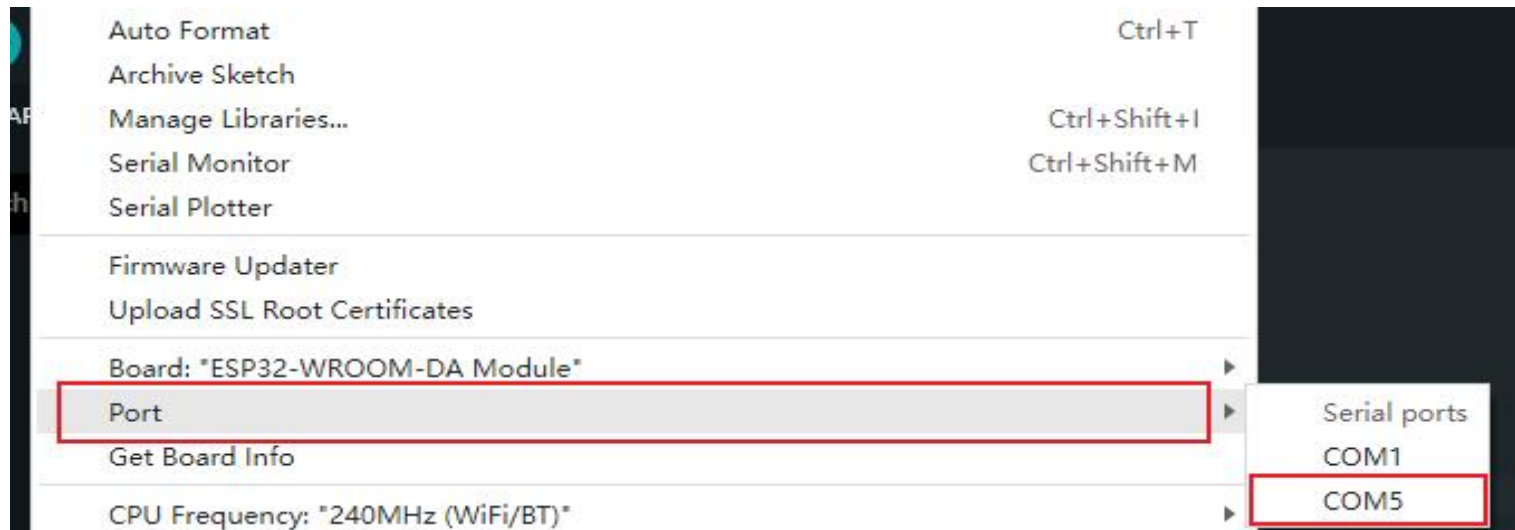


You've saved a copy of your "Flash" in your sketchbook, and if you want to find it again, just open it using the "File > Open Recent" menu option.



Connect the ESP32 board to the computer using a USB cable and check that the Board Type and Serial Port settings are correct

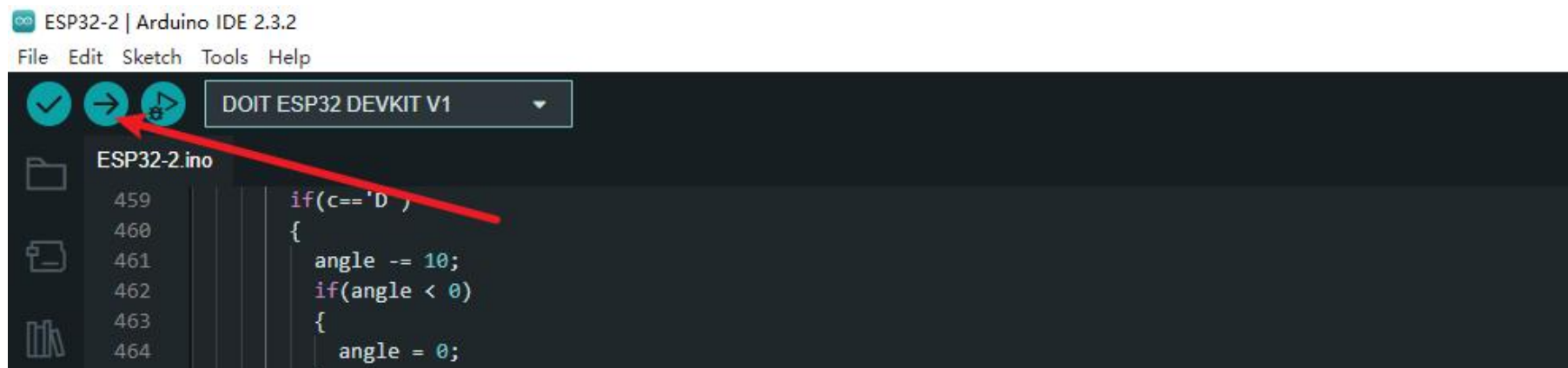




Note: The board type here is ESP32 DEVKIT V1 and the serial port is **COM5**.

The actual serial port appears differently for everyone, although COM5 is selected here, it could be COM3 or COM4 on your computer. A correct COM port should be COM X compliant.

After clicking the "Upload" button, the program will start uploading. At this time, the LED on the ESP32 will start flashing as the sketch is transferred.





```
Using library ESP32Servo at version 3.0.5 in folder: E:\工作\课程设计\Arduino_file\libraries\ESP32Servo
Using library BluetoothSerial at version 2.0.0 in folder: C:\Users\Administrator\AppData\Local\Arduino15\packages\esp32\hardware\esp32\2.0.12\libraries\BluetoothSerial
"C:\Users\Administrator\AppData\Local\Arduino15\packages\esp32\tools\xtensa-esp32-elf-gcc\esp-2021r2-patch5-8.4.0\bin\xtensa-esp32-elf-size" -A "C:\Users\Administrator\AppData\Local\Temp\arduino\sketches\9B54E0FB643161932
Sketch uses 1060849 bytes (80%) of program storage space. Maximum is 1310720 bytes.
Global variables use 39060 bytes (11%) of dynamic memory, leaving 288620 bytes for local variables. Maximum is 327680 bytes.
"C:\Users\Administrator\AppData\Local\Arduino15\packages\esp32\tools\esptool_py\4.5.1/esptool.exe" --chip esp32 --port "COM1" --baud 921600 --before default_reset --after hard_reset write_flash -z --flash_mode dio --flash_freq 80m --flas
esptool.py v4.5.1
Serial port COM1
Connecting.....

A fatal error occurred: Failed to connect to ESP32: No serial data received.
For troubleshooting steps visit: https://docs.espressif.com/projects/esptool/en/latest/troubleshooting.html
Failed uploading: uploading error: exit status 2
```

Upload error: Failed uploading: uploading error: exit status 2

COPY ERROR MESSAGES

Downloading index: package_esp32_index.json

During the "Compile Sketch.." process, you may receive the error message:

This may mean that your board is not connected at all, or that the CP210X driver did not install automatically (if needed) or that the wrong serial port was selected. If you encounter this situation, please check your IDE settings and motherboard connection, take a screenshot and seek help from sales personnel. Or please go to the official website to download the driver yourself. The download website is as follows:

<https://www.silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers>

Once the upload is complete, the board LED should reboot and start flashing. Note that a large portion of this sketch consists of notes. These are not actual program instructions; instead, they simply explain how to make the program work. They are there for your ease of reading. Everything between `/*` and `*/` at the top of the sketch is a block comment that explains the purpose of the sketch.

Single-line comments begin with `//` and everything up to the end of the line is considered a comment.

The first part of the code is:



```
25 // the setup function runs once when you press reset or power the board
26 void setup() {
27     // initialize digital pin LED_BUILTIN as an output.
28     pinMode(LED_BUILTIN, OUTPUT);
29 }
30
```

Every sketch requires a "setup" function, which is the "Void setup()" function, which is executed when the reset button is pressed. It is executed whenever the board resets for any reason, such as first power-up or after uploading a sketch.

The next step is to name the pin and set the output. Here, set "LED_BUILTIN" as the output port. On most ESP32s, including DEVKIT V1, pin 2 is the pin corresponding to the LED. To facilitate programming, the program has set the LED_BUILTIN variable to this pin, so there is no need to rename pin 2 for direct use.

The sketch must also have a "loop" function. Unlike the "Set" function which only runs once, after a reset the "Loop" function will start again as soon as it finishes running the command.

```
// the loop function runs over and over again forever
void loop() {
    digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage
    level)delay(1000);                // wait for a second
    digitalWrite(LED_BUILTIN, LOW);  // turn the LED off by making the voltage LOW
    delay(1000);                      // wait for a second
}
```

Inside the loop function, the command first turns the LED pin on (high), then "delays for 1000 milliseconds (1 second), then turns off the LED pin and pauses for one second. You now want to make your LED blink faster. As As you might have guessed, the key is to change the parameters in "delay()".



```
void loop() {  
  digitalWrite(LED_BUILTIN, HIGH);    // turn the LED on (HIGH is the voltage level)  
  delay(1000);                        // wait for a second  
  digitalWrite(LED_BUILTIN, LOW);     // turn the LED off by making the voltage LOW  
  delay(1000);                        // wait for a second  
}
```

This delay time is in milliseconds, so if you want the "LED" to flash twice as fast, change the value from "1000" to "500". This will pause for half a second on each delay, instead of one second. Upload the sketch again and you should see the "LED" start blinking faster.

At this point, you have understood and mastered the basic Arduino programming knowledge and the basic steps for downloading the program, which lays a good foundation for learning complex projects later.

Note: Sometimes the reason for unsuccessful downloading of the program is voltage problem or the main control board does not enter the download mode. At this time, it is necessary to turn on the power of the battery box and press the motor Boot button continuously while burning the program to make the main control board enter the download mode.

4 Motor test

4.1 Course Introduction

In the previous lesson, we learned the basic method of uploading programs. In this lesson, we will learn



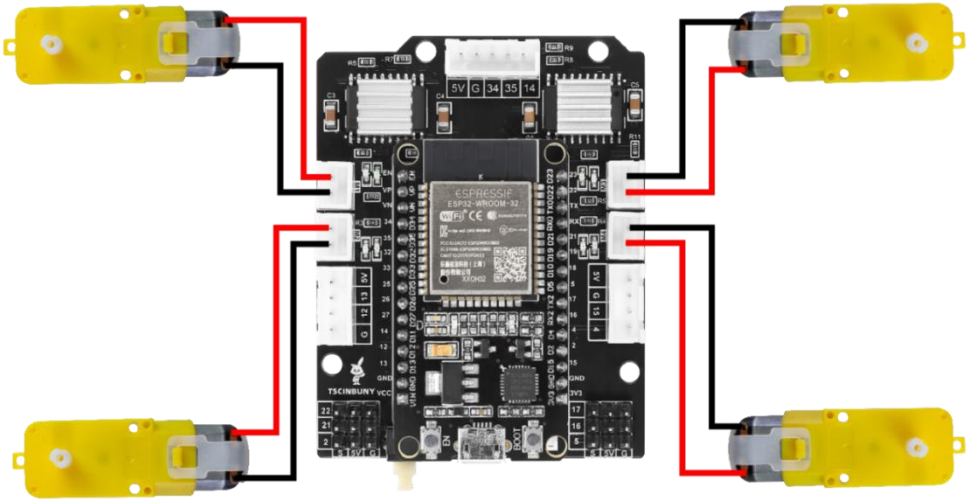
and master the control principles of car movement.

4.2 How the L293D motor driver chip works

L293D determines the rotation direction of the motor by controlling the high and low levels of the input ports (IN1, IN2, IN3, IN4), and cooperates with the enable signal (EN1/2, EN3/4) to start and stop the motor.

EN1/2	IN1	IN2	电机状态
高	高	低	正转
高	低	高	反转
高	高	高	刹车（快速停止）
高	低	低	自由停止
低	X	X	停止

4.3 Wiring diagram





4.4 Code analysis

Open the code file (path:\2_Code-code file\1_Motor_test\1_Motor_test.ino)

Define motor pins:

```
1 //右
2 #define RightFrontFWD 23
3 #define RightFrontBWD 27
4 #define RightBackFWD 33
5 #define RightBackBWD 32
6
7 //左
8 #define LeftFrontFWD 18
9 #define LeftFrontBWD 19
10 #define LeftBackFWD 26
11 #define LeftBackBWD 25
```

Define motor pin mode:

```
void setup() {
    // put your setup code here, to run once:
    pinMode(RightFrontFWD, OUTPUT);
    pinMode(RightFrontBWD, OUTPUT);
    pinMode(RightBackFWD, OUTPUT);
    pinMode(RightBackBWD, OUTPUT);
    pinMode(LeftFrontFWD, OUTPUT);
    pinMode(LeftFrontBWD, OUTPUT);
    pinMode(LeftBackFWD, OUTPUT);
    pinMode(LeftBackBWD, OUTPUT);
}
```




The loop function `loop()` repeatedly executes forward, backward, left turn, and right turn for 3 seconds each. Of course, you can also modify 2000 to 4000 to see the effect.

```
void loop() {
  digitalWrite(RightFrontBWD, motorSpeed); // 右前, 前转, 使用PWM控制速度
  digitalWrite(RightFrontFWD, 0);           // 如果RightFrontFWD不是PWM引脚, 则设为低电平
  digitalWrite(RightBackBWD, 0);           // 右后, 前转, 使用PWM控制速度
  digitalWrite(RightBackFWD, motorSpeed);   // 如果RightBackBWD不是PWM引脚, 则设为低电平

  digitalWrite(LeftFrontFWD, motorSpeed);  // 左前, 前转, 使用PWM控制速度
  digitalWrite(LeftFrontBWD, 0);           // 如果LeftFrontBWD不是PWM引脚, 则设为低电平
  digitalWrite(LeftBackBWD, 0);           // 左后, 前转, 使用PWM控制速度
  digitalWrite(LeftBackFWD, motorSpeed);   // 如果LeftBackBWD不是PWM引脚, 则设为低电平
  delay(3000);

  digitalWrite(RightFrontFWD, motorSpeed); // 右前轮, 后退方向, 利用PWM进行速度调控
  digitalWrite(RightFrontBWD, 0);          // 若RightFrontBWD不支持PWM, 则设为低电平
  digitalWrite(RightBackFWD, 0);          // 右后轮, 后退方向, 利用PWM进行速度调控
  digitalWrite(RightBackBWD, motorSpeed);  // 若RightBackFWD不支持PWM, 则设为低电平

  digitalWrite(LeftFrontBWD, motorSpeed);  // 左前轮, 后退方向, 利用PWM进行速度调控
  digitalWrite(LeftFrontFWD, 0);           // 若LeftFrontFWD不支持PWM, 则设为低电平
  digitalWrite(LeftBackFWD, 0);           // 左后轮, 后退方向, 利用PWM进行速度调控
  digitalWrite(LeftBackBWD, motorSpeed);   // 若LeftBackFWD不支持PWM, 则设为低电平
  delay(3000);

  digitalWrite(RightFrontBWD, motorSpeed); // 右前轮, 前退方向, 利用PWM进行速度调控
  digitalWrite(RightFrontFWD, LOW);        // 若RightFrontFWD不支持PWM, 则设为低电平
  digitalWrite(RightBackFWD, motorSpeed);  // 右后轮, 前进方向, 利用PWM进行速度调控
  digitalWrite(RightBackBWD, LOW);         // 若RightBackBWD不支持PWM, 则设为低电平

  digitalWrite(LeftFrontBWD, motorSpeed);  // 左前轮, 后进方向, 利用PWM进行速度调控
  digitalWrite(LeftFrontFWD, LOW);         // 若LeftFrontBWD不支持PWM, 则设为低电平
  digitalWrite(LeftBackBWD, motorSpeed);   // 左后轮, 后退方向, 利用PWM进行速度调控
  digitalWrite(LeftBackFWD, LOW);         // 若LeftBackFWD不支持PWM, 则设为低电平
  delay(3000);
}
```



5 Follow car

5.1 Course Introduction

Ultrasonic distance measurement is a very useful and widely used measurement method. This lesson mainly understands the working principle of the ultrasonic module, masters the connections of the ultrasonic circuit diagram, and learns the method of ultrasonic module distance measurement through programming.

5.2 Component introduction

HC-SR04 Ultrasonic Module:



Working voltage	DC-5V
Working current	15mA



Working frequency	40KHz
Distance range	2cm~400cm
Measuring angle	15°
Input trigger signal	10 US TTL pulse
Output echo signal	Output TTL level signal, proportional to the range

5.3 How does ultrasound work?

1. The transmitter (trig pin) sends a signal: high-frequency sound;
2. When the signal hits an object, it will be reflected;
3. Receiver (echo pin): Receives the signal reflected back from it.

There are many methods of ultrasonic distance measurement. The principle of this system in ultrasonic measurement is: detect the transmission time of ultrasonic waves from the ultrasonic transmitter through the gas medium to the receiver, multiply this time by the speed of sound in the air, and get the sound propagation distance.

The ultrasonic transmitter emits ultrasonic waves in a certain direction, and the MCU starts timing at the same time. The ultrasonic waves are launched in the air and return immediately when encountering obstacles on the way. The ultrasonic receiver stops timing immediately after receiving the reflected waves. According to the time T recorded by the timer, the distance (s) from the launch point to the obstacle can be calculated.



formula: $S = VT/2$

Four factors limit the maximum measurable distance of an ultrasound system: the amplitude of the ultrasound wave, the texture of the reflector, the angle between the reflected and incident sound waves, and the sensitivity of the receiving transducer. The ability of the receiving transducer to directly receive the acoustic pulse will determine the minimum measurable distance.

5.4 Principle of following the car

If it is judged that the ultrasonic detection distance directly ahead is greater than 25cm, the car will move forward, and the car will stop when the distance is between 15cm and 25cm. Finally, when the ultrasonic distance is too close and less than 15cm, the car will move backwards.

5.5 Code analysis

Open the code file (path:\2_Code-code file\2_Ultrasonic_follow\2_Ultrasonic_follow.ino)

Define ultrasonic sensor pins

```
//超声波  
#define Trigpin 13  
#define Echopin 12
```

Initialize the ultrasonic sensor and motor pins.



```
pinMode(Trigpin,OUTPUT);
pinMode(Echopin,INPUT);

pinMode(RightFrontFWD, OUTPUT);
pinMode(RightFrontBWD, OUTPUT);

pinMode(LeftFrontFWD, OUTPUT);
pinMode(LeftFrontBWD, OUTPUT);
```

Ultrasonic ranging function

```
unsigned long measureDistance() {
    digitalWrite(Trigpin, HIGH);
    delayMicroseconds(20);
    digitalWrite(Trigpin, LOW);
    delayMicroseconds(10);
    unsigned long Time_Echo_us = pulseIn(Echopin, HIGH, 60000); // 设置超时时间为60000微秒
    if(Time_Echo_us > 1) {
        return (Time_Echo_us * 34 / 100) / 2;
    } else {
        return 0; // 如果读取失败，返回0
    }
}
```

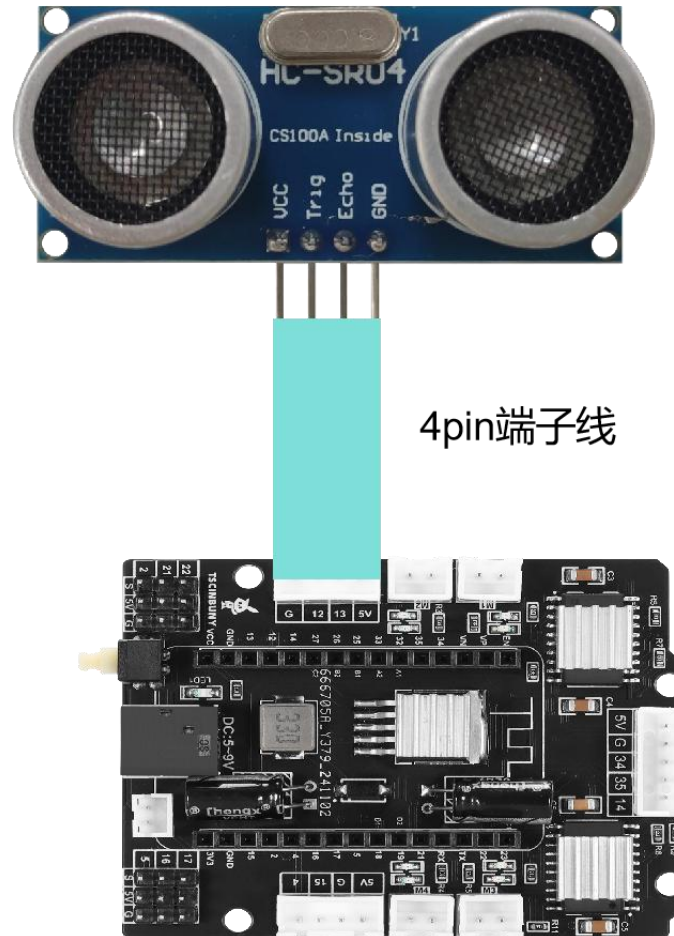
Car motion function (collapsed)



```
void moveForward(int motorSpeed) {  
    //前进  
    digitalWrite(RightFrontBWD, motorSpeed); // 右前, 前转, 使用PWM控制速度  
    digitalWrite(RightFrontFWD, 0);          // 如果RightFrontFWD不是PWM引脚, 则设为低电平  
    digitalWrite(RightBackBWD, 0);           // 右后, 前转, 使用PWM控制速度  
    digitalWrite(RightBackFWD, motorSpeed);  // 如果RightBackBWD不是PWM引脚, 则设为低电平  
  
    digitalWrite(LeftFrontFWD, motorSpeed);  // 左前, 前转, 使用PWM控制速度  
    digitalWrite(LeftFrontBWD, 0);           // 如果LeftFrontBWD不是PWM引脚, 则设为低电平  
    digitalWrite(LeftBackBWD, 0);           // 左后, 前转, 使用PWM控制速度  
    digitalWrite(LeftBackFWD, motorSpeed);   // 如果LeftBackBWD不是PWM引脚, 则设为低电平  
}  
  
void moveBackward(int motorSpeed) { ...  
}  
  
unsigned long measureDistance() { ...  
}
```



5.6 Wiring diagram





6 obstacle avoidance car

6.1 Course Introduction

In the previous course, we have understood and learned the relevant knowledge of ultrasonic sensors and the working principle of the following car. In this course, we will complete an automatic obstacle avoidance car.

6.2 Principle of obstacle avoidance

The car's ultrasonic detection distance is less than or equal to 15cm in front (that is, when it encounters an obstacle). The car will back up and turn left to avoid the obstacle in front.

6.3 Code upload

Open the code file

(path: \2_Code-code file\Ultrasonic_obstacle_avoidance\Ultrasonic_obstacle_avoidance.ino)

The sensor pins and ultrasonic distance acquisition functions in the code are consistent with the previous lesson, and a left turn function is added to the movement function.



```
void rotateLeft(int motorSpeed) {  
    //左转  
    digitalWrite(RightFrontBWD, motorSpeed); // 右前轮, 前退方向, 利用PWM进行速度调控  
    digitalWrite(RightFrontFWD, LOW); // 若RightFrontFWD不支持PWM, 则设为低电平  
    digitalWrite(RightBackFWD, motorSpeed); // 右后轮, 前进方向, 利用PWM进行速度调控  
    digitalWrite(RightBackBWD, LOW); // 若RightBackBWD不支持PWM, 则设为低电平  
  
    digitalWrite(LeftFrontBWD, motorSpeed); // 左前轮, 后退方向, 利用PWM进行速度调控  
    digitalWrite(LeftFrontFWD, LOW); // 若LeftFrontBWD不支持PWM, 则设为低电平  
    digitalWrite(LeftBackBWD, motorSpeed); // 左后轮, 后退方向, 利用PWM进行速度调控  
    digitalWrite(LeftBackFWD, LOW); // 若LeftBackFWD不支持PWM, 则设为低电平  
}
```

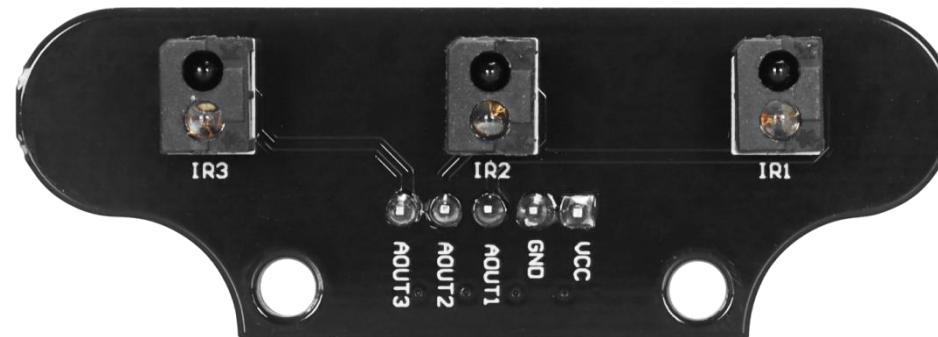
When the obstacle is less than 15cm away from the car, the car stops suddenly, then retreats, and finally turns left to avoid the obstacle.

```
void loop() {  
    unsigned long Len_mm = measureDistance();  
    Serial.print(Len_mm, DEC);  
    Serial.println("mm");  
    delay(100);  
    if(Len_mm >= 150) moveForward(motorSpeed);  
    if(Len_mm < 150)  
    {  
        stopMoving();  
        Serial.print("Len_mm: ");  
        Serial.println(Len_mm);  
  
        moveBackward(motorSpeed); //后退  
        delay(800);  
        rotateLeft(motorSpeed); //左移  
        delay(1500);  
        stopMoving();  
    }  
}
```



7 tracking car

7.1 Three-way tracking module



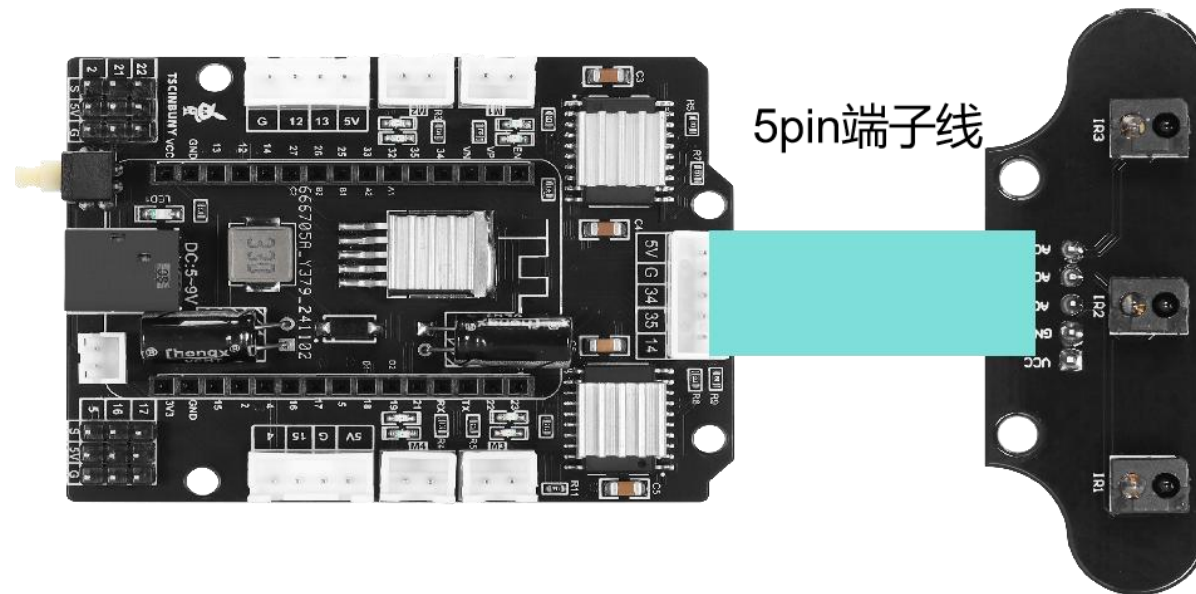
How does the three-channel tracking module work?

This sensor belongs to infrared reflection detection, which uses different colors of infrared light objects to have different reflection intensity characteristics. When the car is walking, it continuously emits infrared light to the ground. When the infrared light encounters the white floor, it is diffusely reflected, and the reflected light is The receiving tube installed in the car receives; if it encounters a black line, the infrared light is absorbed, and the receiving tube in the car does not receive the infrared ray. Therefore,



the position of the black line and the driving route of the car can be determined based on whether the reflected infrared light is received.

7.2 Tracking module wiring diagram





7.3 Code analysis

Open the code file (path: 2_Code-code file\4_Infrared_patrol_line\4_Infrared_patrol_line.ino)

Define three tracking sensor variables and set pins as inputs

```
Infrared_patrol_line.ino
1  #define LEFT_LINE_TRACJING    34
2  #define CENTER_LINE_TRACJING  35
3  #define right_LINE_TRACJING   14
4  //右
```

```
63  Serial.begin(115200);
64  pinMode(LEFT_LINE_TRACJING, INPUT);
65  pinMode(CENTER_LINE_TRACJING, INPUT);
66  pinMode(right_LINE_TRACJING, INPUT);
67
```

The loop function body obtains the detection information of the 3-channel tracking module and performs vehicle motion control based on the detection results (high level means black line is detected).

In the loop function loop(), save the values of the three sensors read into variables.



```
void loop() {  
  // put your main code here, to run repeatedly:  
  Left_Tra_Value = analogRead(LEFT_LINE_TRACJING);  
  Center_Tra_Value = analogRead(CENTER_LINE_TRACJING);  
  Right_Tra_Value = analogRead(right_LINE_TRACJING);  
}
```

After the code is uploaded, we open the serial monitor to check the values read by the three infrared sensors. Normally, the sensor that detects the black line reads more than 3500. When we see the following values, it means that the LEFT sensor detects the black line, and the other The sensor does not detect the black line.

Left	Center	Right
4095	150	126
4095	151	124
4095	150	126
4095	147	121
4095	144	121
4095	144	122
4095	146	122
4095	150	121



The code for placing the car on the black route and controlling the car to implement the tracking function is as follows:

```
if (Left_Tra_Value >= Black_Line && Right_Tra_Value < Black_Line)
{
    int motorSpeed = 1;
    moveForward(motorSpeed);
}

if (Left_Tra_Value >= Black_Line && Left_Tra_Value > Right_Tra_Value)
{
    rotateLeft(motorSpeed); //左移
}

if (Right_Tra_Value >= Black_Line && Right_Tra_Value > Left_Tra_Value)
{
    rotateRight(motorSpeed); //右移
}

if (Left_Tra_Value >= Black_Line && Center_Tra_Value >= Black_Line && Right_Tra_Value >= Black_Line)
{
    stopMoving();
}

if (Left_Tra_Value < Black_Line && Center_Tra_Value < Black_Line && Right_Tra_Value < Black_Line)
{
    stopMoving();
}

delay(100);
```



8 Servo mechanical arm

8.1 Course Introduction

This section mainly focuses on understanding the properties and characteristics of the servo, learning related knowledge about the servo, mastering the debugging methods and circuit connections of the servo, and finally experiencing the working mode of the servo in Arduino programming.

8.2 Introduction to steering gear



The MG90S servo motor control pulse signal period is 20MS pulse width modulation signal (PWM), the pulse width is from 0.5ms to 2.5ms, and the corresponding steering position changes linearly from 0 to 180 degrees.



In other words, if a certain pulse width is provided to the servo, its output shaft will maintain a certain corresponding angle. No matter how the external torque changes, it will not change the output angle to a new corresponding position until another pulse signal is provided to it.

There is a reference circuit inside the steering gear, which generates a pulse signal with a period of 20ms and a width of 1.5ms. There is a comparator that compares the external signal with the reference signal to determine the direction and size, thereby generating a motor rotation signal.

8.3 Code analysis

Open the code file (path: 2_Code\Servo_Angle\Servo_Angle.ino)

Import the ESP32Servo library file and declare the signal ports connected to the three servo motors as 2/21/22.

```
1  #include <ESP32Servo.h>
2
3  Servo myServo; // 创建一个Servo对象
4  Servo myServo1; // 创建一个Servo对象
5  Servo myServo2; // 创建一个Servo对象
6
```



```
10 void setup() {  
11     // put your setup code here, to run once:  
12     myServo.attach(2); // 将舵机的信号线连接到ESP32的GPIO  
13     myServo1.attach(21); // 将舵机的信号线连接到ESP32的GPIO  
14     myServo2.attach(22); // 将舵机的信号线连接到ESP32的GPIO  
15 }
```

The cycle is executed, and the turntable servo motor rotates back and forth from 140° to 10° .

```
19     for ( angle = 140; angle >= 10; angle -= 1)  
20     {  
21         myServo.write(angle);  
22         delay(15);  
23     }  
24     for (angle = 10 ; angle<= 140; angle += 1)  
25     {  
26         myServo.write(angle);  
27         delay(15);  
28     }
```

Cycle execution, the robot arm servo motor rotates back and forth from 170° to 10° .



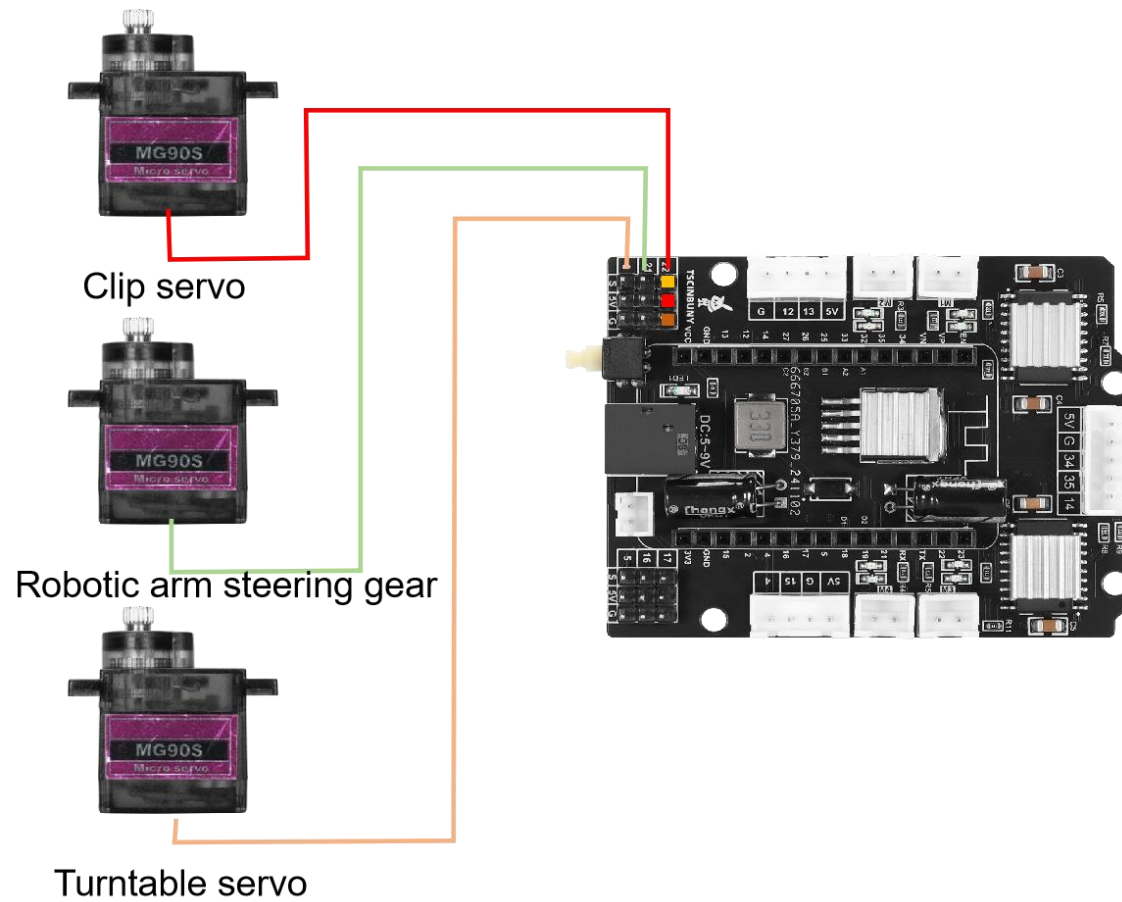
```
30     for (angle1 = 10; angle1 <= 170; angle1 += 1)
31     {
32         myServo1.write(angle1);
33         delay(15);
34     }
35
36     for (angle1 = 170; angle1 >= 10; angle1 -= 1)
37     {
38         myServo1.write(angle1);
39         delay(15);
40     }
```

The cycle is executed, and the clamp servo motor rotates back and forth from 60° to 0° .

```
42     for (angle2 = 60; angle2 >= 0; angle2 -= 1)
43     {
44         myServo2.write(angle2);
45         delay(15);
46     }
47     for (angle2 = 0; angle2 <= 60; angle2 += 1)
48     {
49         myServo2.write(angle2);
50         delay(15);
51     }
```



8.4 Wiring diagram





9 Comprehensive control

9.1 Course Introduction

In the second lesson, we have mastered how to set up the APP. This lesson mainly understands the programming principles of smart Bluetooth control.

9.2 Bluetooth comprehensive control code analysis

The basic principle of Bluetooth-controlled smart car is to send a string to the car through the app.

After the car receives the string, it determines the type of the string.

Change the movement mode of the car

The default is in "free control" mode, which is `Mode_var = 1`, and we have set four other modes.

When the Bluetooth APP presses the OA button, it sends "2"



At this time, "1" is received through the "Switch...case" judgment in the code, and 2 is saved into the mode variable. At this time, "Car_mode = 2".

When Car_mode = 2, continue to detect the signal sent by Bluetooth

```
case '2':  
    myServo3.write(downServoMidAngle);  
    model_var = 2;  
    pCharacteristic->setValue("model_var = 2");  
    break;
```

In the loop function, when the car_mode variable is 2, the obstacle avoidance method is executed.

```
if(model_var == 2){  
    ObstacleAvoidance();  
}
```

In the same way, you can understand the programming principles of other modes. When "3" is received, it is follow mode, when "4" is received, it is tracking mode.

Follow the method:



```
if(UT_distance < 150)
{
    moveBackward(motorSpeed);
}

if(150 <= UT_distance && UT_distance <= 250)
{
    stopMoving();
}

if(UT_distance >= 250)
{
    moveForward(motorSpeed);
}
```

tracing method

```
if (Left_Tra_Value < Black_Line && Center_Tra_Value >= Black_Line && Right_Tra_Value < Black_Line)
{
    myServo3.write(downServoMidAngle);
    moveForward();
}

if (Left_Tra_Value >= Black_Line && Left_Tra_Value > Right_Tra_Value)
{
    myServo3.write(downServoMidAngle + 20);
    TrackLeft();//左循迹
}
```



```
if (Right_Tra_Value >= Black_Line && Right_Tra_Value > Left_Tra_Value)
{
    myServo3.write(downServoMidAngle - 20);
    TrackRight();//右循迹
}

if (Left_Tra_Value >= Black_Line && Center_Tra_Value >= Black_Line && Right_Tra_Value >= Black_Line)
{
    stopMoving();
}

if (Left_Tra_Value < Black_Line && Center_Tra_Value < Black_Line && Right_Tra_Value < Black_Line)
{
    stopMoving();
}
```

Robotic arm method

```
void arm_control(){
    while(AngleI)
    {
        angle1 += 10;
        if(angle1 > 180)
        {
            angle1 =180;
        }
        myServo1.write(angle1);
        delay(200); // 等待舵机到达指定位置
        Serial.print("Angle1: ");
        Serial.println(angle1);
    }
}
```

At this point, you have completed the entire tutorial!